ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

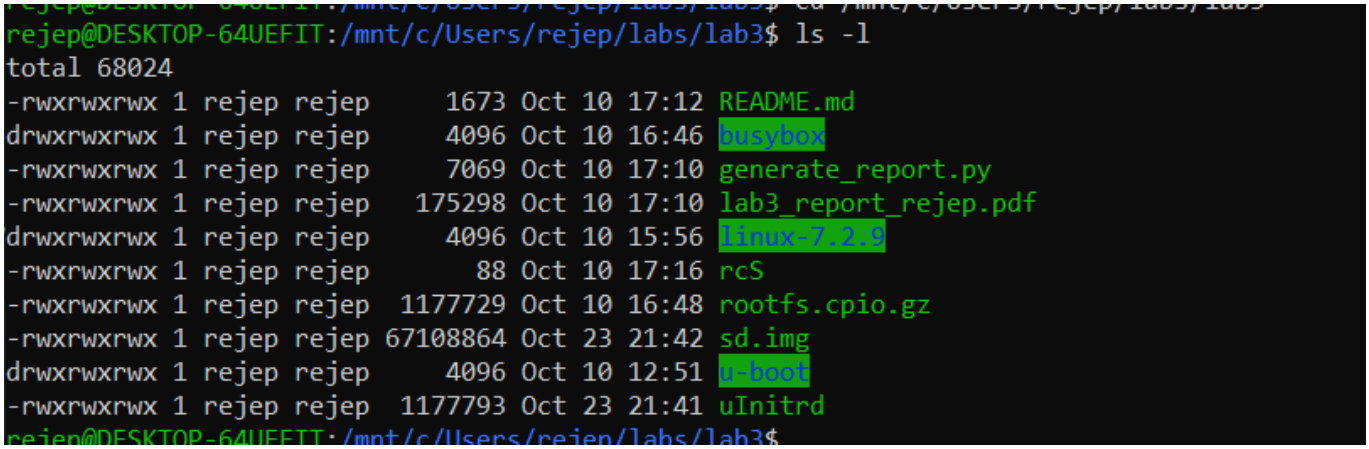
Тема: Сборка компонентов встроенной системы для архитектуры ARM

Выполнил: Реджеп

Часть 1. Настройка среды и загрузка исходных кодов

Шаг 1. Развертывание рабочего окружения проекта

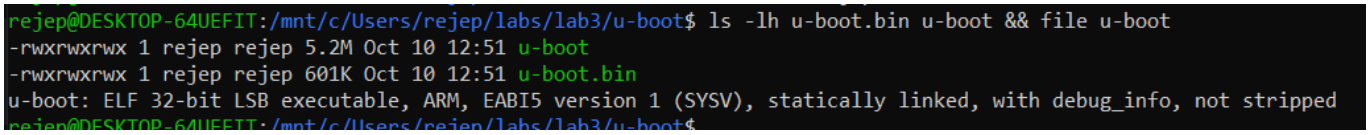
Была инициализирована рабочая директория лабораторной работы, установлены эмулятор QEMU ARM, базовые утилиты сборки и кросс-компилятор arm-linux-gnueabi-. Загружены официальные исходные коды стабильного ядра Linux (версия 7.2.9), загрузчика первого этапа U-Boot и минимального UNIX-инструментария BusyBox.



Часть 2. Конфигурирование, компиляция и запуск системы

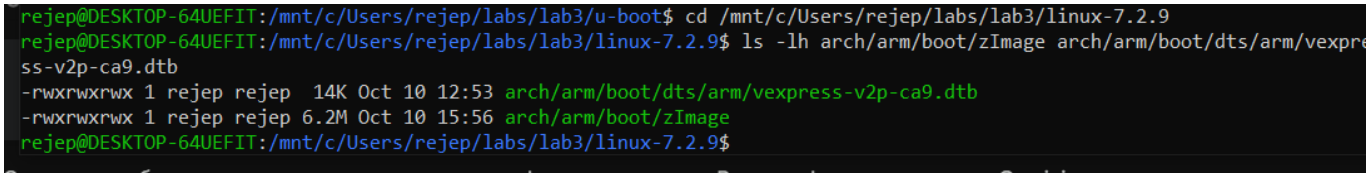
Шаг 2. Сборка первого этапа загрузки (U-Boot)

Выполнена конфигурация под плату Versatile Express (vexpress_ca9x4_defconfig) и осуществлена кросс-компиляция загрузчика. В результате успешно сгенерирован бинарный исполняемый файл u-boot архитектуры ARM.



Шаг 3. Кросс-компиляция стабильного ядра Linux

Ядро Linux версии 7.2.9 было сконфигурировано под архитектуру ARM (vexpress_defconfig) и собрано. Сформирован сжатый образ ядра zImage и файлы дерева устройств (DTB).



Шаг 4. Сборка автономной файловой системы BusyBox

Для корректной работы утилит в изолированном пространстве QEMU была произведена статическая линковка компонентов BusyBox (CONFIG_STATIC=y) с отключением конфликтующих с современным дистрибутивом модулей. Проверка бинарника утилитой file подтверждает статическую линковку под ARM.

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ file busybox/_install/bin/busybox
busybox/_install/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, BuildID[sha1]=4
c80e35e2109aa8ae3e0930bca85dca1b47fface, for GNU/Linux 3.2.0, stripped
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$
```

Шаг 5. Запуск загрузчика U-Boot и инициализация компонентов

Все скомпилированные компоненты (zImage, DTB и упакованный initramfs uInitrd) были помещены на виртуальный образ SD-карты (sd.img). Эмулятор QEMU запускает загрузчик U-Boot, который обеспечивает интерактивную среду загрузки.

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -kernel u-boot
/u-boot -sd sd.img -serial stdio -display none
WARNING: Image format was not specified for 'sd.img' and probing guessed raw.
         Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
         Specify the 'raw' format explicitly to remove the restrictions.
[W][00:04:04.401655] pw.conf      | [          conf.c: 1182 try_load_conf()] can't load config client.conf: No such file
 or directory
[E][00:04:04.402175] pw.conf      | [          conf.c: 1215 pw_conf_load_conf_for_context()] can't load config client.co
nf: No such file or directory
pulseaudio: set_sink_input_volume() failed
pulseaudio: Reason: Invalid argument
pulseaudio: set_sink_input_mute() failed
pulseaudio: Reason: Invalid argument


U-Boot 2026.10-g837387a5aebc (Oct 10 2026 - 12:49:28 +0300)

DRAM:  512 MiB
Core:  23 devices, 11 uclasses, devicetree: embed
Flash: 128 MiB
MMC:   mmci@5000: 0
*** Warning - bad CRC, using default environment

In:    uart@9000
Out:   uart@9000
Err:   uart@9000
Net:   eth0: ethernet@3,02000000
Hit any key to stop autoboot: 0
no mmc device at slot 1
switch to partitions #0, OK
```

```
=> fatload mmc 0 0x60000000 zImage
6441848 bytes read in 9026 ms (696.3 KiB/s)
=> fatload mmc 0 0x61000000 vexpress-v2p-ca9.dtb
14329 bytes read in 34 ms (411.1 KiB/s)
=> fatload mmc 0 0x62000000 uInitrd
1177793 bytes read in 1720 ms (668 KiB/s)
=> setenv bootargs "console=ttyAMA0 root=/dev/ram0 rdinit=/sbin/init"
=> bootz 0x60000000 0x62000000 0x61000000
Kernel image @ 0x60000000 [ 0x000000 - 0x624b78 ]
## Loading init Ramdisk from Legacy Image at 62000000 ...
   Image Name:
   Image Type:   ARM Linux RAMDisk Image (gzip compressed)
   Data Size:    1177729 Bytes = 1.1 MiB
   Load Address: 00000000
   Entry Point:  00000000
   Verifying Checksum ... OK
## Flattened Device Tree blob at 61000000
   Booting using the fdt blob at 0x61000000
Working FDT set to 61000000
   Loading Ramdisk to 7e9f7000, end 7eb16881 ... OK
   Loading Device Tree to 7e9f0000, end 7e9f67f8 ... OK
Working FDT set to 7e9f0000

Starting kernel ...
```

Шаг 6. Верификация работающей RootFS

После завершения стадий инициализации управление передается процессу /sbin/init, который отрабатывает сценарий etc/init.d/rcS и активирует командную строку. С помощью команд uname -a и ls -l верифицировано развертывание кастомной ОС Linux 7.2.9.

```
~ # uname -a
Linux (none) 7.2.9 #1 SMP Sat Oct 10 15:51:25 MSK 2026 armv7l GNU/Linux
~ # ls -l
total 0
drwxrwxrwx    2 1000     1000             0 Oct 10 13:46 bin
drwxr-xr-x    4 0        0             2920 Oct 23 21:05 dev
drwxrwxrwx    3 1000     1000             0 Oct 10 13:48 etc
lrwxrwxrwx    1 1000     1000            11 Oct 10 13:46 linuxrc -> bin/busybox
dr-xr-xr-x   77 0        0                0 Oct 23 21:05 proc
drwx------    2 0        0                0 Oct 10 09:54 root
drwxrwxrwx    2 1000     1000             0 Oct 10 13:46 sbin
dr-xr-xr-x   12 0        0                0 Oct 23 21:05 sys
drwxrwxrwx    4 1000     1000             0 Oct 10 13:46 usr
~ #
```

Шаг 7. Анализ активных файловых систем и подтверждение цепочки загрузки

Вызов команды mount внутри запущенной ОС подтверждает успешное завершение каноничной цепочки загрузки. Строка 'rootfs on /' доказывает, что корневой раздел развернут в RAM-диске из подготовленного архива initramfs. Наличие смонтированных виртуальных директорий /proc, /sys и /dev подтверждает корректную отработку стартового скрипта инициализации.

```
~ # mount
rootfs on / type rootfs (rw)
none on /proc type proc (rw,relatime)
none on /sys type sysfs (rw,relatime)
none on /dev type devtmpfs (rw,relatime,size=243236k,nr_inodes=60809,mode=755)
~ #
```

Заключение

В ходе лабораторной работы были успешно освоены принципы проектирования Embedded Linux систем. Все компоненты (U-Boot, Ядро, initramfs/RootFS) были скомпилированы из исходных кодов с помощью кросс-компилятора и объединены в каноничную цепочку загрузки (U-Boot -> zImage -> uInitrd -> RootFS) на виртуальной плате ARM Versatile Express под управлением QEMU, продемонстрировав полную интерактивную работоспособность.